



German Driving-School Chatbot

A production-grade, bilingual (German and English) assistant that answers pricing and enrollment questions with verified accuracy. This document explains what the system does, why it is designed the way it is, and how every part works.

44 / 44	0	80	DE / EN
Price sheets verified against source PDFs	Invented prices under adversarial testing	Automated tests passing	Automatic bilingual conversation

Repository: `RagPipelineChatBot-GermanDataset`

Architecture: `Deterministic price core + bounded language layer`

Deployment: `Docker, data mounted privately, never baked into the image`

Table of contents

01	Executive summary	
02	The problem and the goal	
03	Key requirements	
04	What the system does	
05	Architecture: the design that guarantees accuracy	
06	How it works, component by component	
07	The "never wrong" guarantees	
08	Testing and quality assurance	
09	Security and confidentiality	
10	Deployment	
11	Technology stack	
12	Repository structure	
13	Results	
14	Future enhancements	
15	Glossary of German terms	

A chatbot built around one non-negotiable principle

A German driving school wanted a customer-facing assistant that could answer pricing and general questions accurately, in the customer's own language, at any hour. Driving-license pricing is exact and safety-critical for the business, so the usual approach of letting a language model generate answers was not acceptable: language models can invent numbers, and a single wrong price shown to a customer is a real business problem.

The solution separates the system into a deterministic core and a language layer. All prices come from a structured, human-verifiable store that is validated against the original documents. The language model is used only to understand messy free text and to translate, never to produce a price. The result is a chatbot that behaves naturally, speaks German and English, asks clarifying questions when needed, delivers the official price sheet as a downloadable PDF, and is provably accurate.

Verified outcomes

- **All 44** official price sheets integrated and checked against their source PDFs, with zero mismatches.
- **100 percent** price accuracy enforced automatically on every code change.
- **Bilingual:** German questions get German answers, English questions get English answers.
- **Guided follow-up questions** for classes with multiple variants (for example Class B).
- **Resistant to adversarial input:** no injection, role-play, or edge case could make it state a wrong or invented price.
- **80 automated tests**, a dedicated evaluation gate, and a one-command Docker deployment.

Many license classes, many variants, one official price each

The client offers a large catalogue of German driving-license classes. Each class can have several variants: for example, a car license can be a standard manual course, an automatic course, a course combined with a trailer entitlement, a conversion of a foreign license, and more. Every variant has its own official price sheet.

The goal was an assistant that:

- **1.** Answers pricing questions with the exact, current price for the correct variant.
- **2.** Asks the right follow-up questions when a request is ambiguous.
- **3.** Answers general questions (opening hours, required documents, registration process).
- **4.** Replies in the language the customer used.
- **5.** Sends the official price sheet as a document or downloadable link.
- **6.** Never gives a wrong answer.

Requirement to implementation, at a glance

REQUIREMENT	HOW IT IS MET
Integrate all pricing documents	44 price sheets parsed into a structured store
Preprocess data before loading into any vector store	A dedicated extraction and verification pipeline runs first
Short-term memory	Each conversation keeps a session with filled-in "slots"
Follow-up questions for pricing	Curated decision trees narrow a class to one variant
German in, German out; English in, English out	Automatic language detection and bilingual replies
Pricing documents delivered as a download	A document endpoint serves the official PDF
Never give a wrong answer	Prices come only from a verified store, validated against the source

Exact prices, guided questions, real answers

- EUR** **Exact pricing with a full breakdown.** For any class and variant it returns the exact total, an itemized breakdown (base fee, lessons, special drives, exam fees), a note about external fees the school does not charge, and a link to the official PDF.
- ?** **Guided follow-up questions.** When a request could mean several variants, the bot asks short, clear questions (for example: new license or special case, car only or combined with another class, manual or automatic) until exactly one variant remains.
- i** **General question answering.** It answers questions such as opening hours, required documents, document validity, simulator availability, and registration steps, using the client's own written material.
- AB** **Bilingual conversation.** It detects German or English automatically and mirrors the language, including a faithful translation of German information into English.
- PDF** **Document delivery.** Every pricing answer includes a downloadable official price sheet.
- !** **Safe fallback.** When it is not confident, it hands the customer off to the team rather than guessing.

Example: a pricing conversation (German)

```
Customer: Was kostet Klasse B?  
Bot:      Is this a new license or a special case? [options]  
Customer: New license  
Bot:      Car only, or combined with another class? [options]  
Customer: Car only  
Bot:      Manual or automatic? [options]  
Customer: Automatic  
Bot:      Shows the exact total for the automatic variant, an itemized  
          breakdown, the external-fee note, and a Download price sheet (PDF) button.
```

Example: a general question (English input)

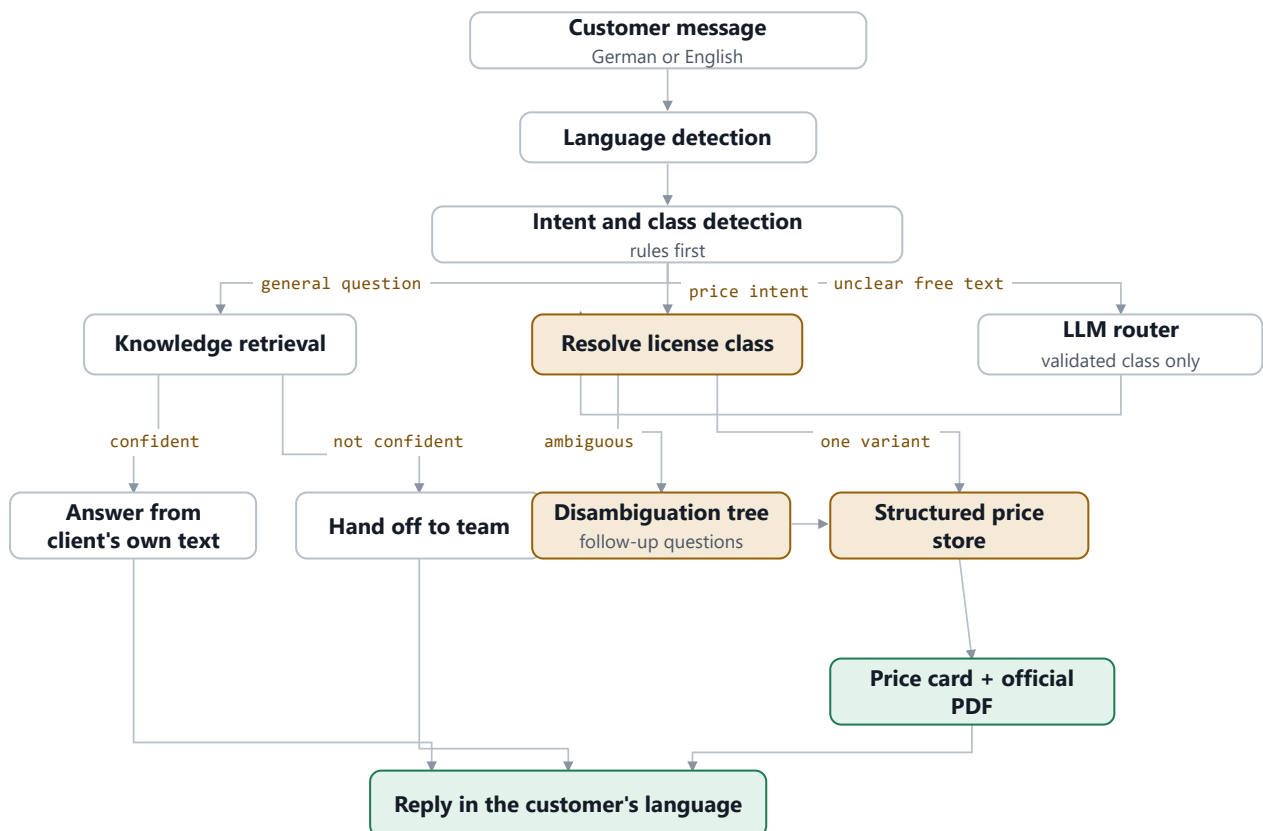
```
Customer: What documents do I need to register?  
Bot:      Identity card or passport; for minors, additional consent of the guardian.
```

The design that guarantees accuracy

The single most important design decision is that the language model never produces a price. The system is split into two cooperating parts.

- **A deterministic core** owns all facts. Prices live in a structured store that was extracted from the official documents and verified against them. Class resolution and the follow-up question logic are rule-based and fully testable.
- **A language layer** helps only with understanding and phrasing. It classifies intent, maps unusual free text to a known class, and translates. It has no authority over facts.

System flow



- ◆ Every euro amount travels the **Resolve** → **Disambiguation/Store** → **Price card** path (accent-colored boxes). It is read verbatim from the verified store and is never generated by the model. A misunderstanding can lead to another question or a hand-off. It can never produce a wrong number.

From raw PDFs to a verified conversation

6.1 Data pipeline (preprocessing before any vector store)

The raw material is a set of official PDF documents: the price sheets, a business information questionnaire, and consultation documents. The pipeline processes these before anything is loaded into a search index.

- **Manifest and taxonomy.** Every file is catalogued (class, variant, date, document type). A taxonomy of German license classes is built. Codes whose meaning is not certain are flagged for client confirmation rather than guessed.
- **Price extraction.** Each price sheet has a very regular layout (an itemized offer with a totals block). A deterministic parser reads every line item and the totals using exact patterns, with no language model involved. It handles real-world quirks such as optional value-added-tax lines, decimal training hours, participant units, and multi-page carry-overs.
- **Automatic self-verification.** Each sheet is internally consistent: the net amount plus value-added tax equals the grand total, and the sum of the line items equals the grand total. The parser checks both. If a value is misread or a line is dropped, the equation breaks and the sheet is flagged. All 44 sheets pass both checks.
- **Knowledge extraction.** The business questionnaire and consultation documents are turned into clean, retrievable passages and question-and-answer pairs.
- **Versioning.** When two generations of a price sheet exist, the newest is marked current and the older is archived so an outdated price is never served.

6.2 Structured price store and query layer

The verified prices are loaded into a small database and exposed through a deterministic query layer with four operations: get the exact price record for a specific variant (archived variants are withheld); list the variants of a base class (used to build follow-up questions); get the downloadable document link for a variant; resolve free text to a license class, or report that it is ambiguous and needs a follow-up.

By design, asking about a bare class that has many variants (for example a car license) returns "ambiguous", which triggers the follow-up questions rather than a guessed answer.

6.3 Disambiguation trees (the follow-up questions)

For each class with multiple variants, a curated decision tree drives the follow-up questions. A short sequence of questions narrows the request to exactly one variant. The trees were derived from the client's own consultation material.

An automated test guarantees that the set of variants reachable through the trees is exactly equal to the set of current variants in the store. This means no price is unreachable and no branch leads to a dead end.

Is this a new license or a special case?

- New license → car only or combined with another class?

- Car only → manual or automatic?
- Special case → conversion, re-issuance, school change, or removing a restriction

6.4 Knowledge base and hybrid retrieval

General questions are answered from the client's own written material, so the answer is always the client's real text and can never be invented. Retrieval uses two methods together: **keyword search** with German-aware normalization, for exact term matches, and **semantic search** (embeddings) for questions phrased differently from the source text.

The two are blended: embeddings propose candidate passages, and keyword scoring re-ranks them so the passage that actually contains the question's terms wins. If the system is not confident, it hands off to the team instead of returning a weak match. This precision-first policy means the bot would rather say "let me connect you to the team" than show a wrong answer.

6.5 Language layer (bounded use of the model)

The language model is used for two narrow tasks: **intent and class routing**, where a proposed class is validated against the known list so the model cannot introduce an unknown class and never returns a price; and **translation**, rendering German information into English with an explicit instruction to preserve every number, time, and address. If no model key is configured or a call fails, the system falls back to its deterministic behaviour. Nothing about pricing depends on the model.

6.6 Dialogue engine (memory and language)

The dialogue engine ties everything together. It detects the language, classifies the intent, routes to pricing or to knowledge, walks the follow-up questions, formats the reply, and hands off when needed. It keeps a short-term memory per session: once the customer has answered a question, that answer is remembered as a filled-in "slot" and is not asked again. Separate conversations are fully isolated from each other.

6.7 API and web widget

A web service exposes the engine over a small set of endpoints, and serves an embeddable chat widget with a German and English toggle, clickable option buttons, an itemized price card, and a Download price sheet button.

METHOD	PATH	PURPOSE
GET	/	The embeddable web chat widget
POST	/api/session	Start a conversation, returns the greeting
POST	/api/message	Send a message or an option choice, returns the next reply
GET	/api/document/{variant}	Download the official price sheet PDF (current only)
GET	/api/health	Liveness and number of price sheets loaded

Accuracy is enforced at every layer, not assumed



Prices are never generated. They are read verbatim from the verified store.



Extraction is self-checked. Net plus tax equals the total, and the line items sum to the total, on all 44 sheets.



Served prices match the source. An automated test re-parses the grand total directly from each source PDF with a second parser and compares it to what the bot serves. Any difference fails the build.



Ambiguity is a stop. A class with several variants is never answered until a single variant is chosen.



Freshness. Only current price sheets are served; superseded sheets are archived.



General answers are grounded. They are the client's own text, gated by a confidence threshold; below it, the bot hands off.



Refusal over guessing. Unknown classes, discount requests, and nonsense never produce a price. They lead to a question or a handoff.

Verified against ground truth, then attacked on purpose

The project is validated at three levels.

Level 1 · Automated tests (80)

Cover the taxonomy parser, the store and its resolution logic, the disambiguation trees, the dialogue flows in both languages, the knowledge retrieval, the language layer (with a fake model, so tests make no live calls), the semantic index, and the web API.

Level 2 · Evaluation gate

A curated set of the client's own example questions plus adversarial probes runs end to end. The price baseline (every current variant equals its signed-off value) and the query cases all pass.

Level 3 · Ground-truth price gate

Every served price is compared to the grand total re-parsed directly from the source PDF. This is the strongest check, because it verifies agreement with the official document itself, not merely internal consistency.

Red team · Adversarial pass

Tested with tricky phrasings, typos, mixed-language input, prompt injection and role-play attempts, empty and huge inputs, emoji, and database-style inputs. Two real defects were found and fixed during that pass, and retrieval quality was improved.

After the fixes, all checks pass: 44 of 44 prices match the source PDFs, and no adversarial input could produce a wrong or invented price.

✓ 44 / 44 prices match source

✓ 80 tests green

✓ 0 fabricated prices under attack

✓ evaluation suite passing

Client data stays private

The client's documents are confidential. They contain business details and, in some documents, bank and tax identifiers. The project treats them accordingly.

- The entire data folder (source PDFs and everything derived from them, including extracted prices) is excluded from version control and never published.
- Only application code, schemas, tests, and documentation are public.
- Secrets (the model API key, the hand-off email, the business name) are read from a local environment file that is excluded from version control and is never committed.
- The client's business name is configured at runtime, not written into the code.
- Every code change is scanned to confirm that no data, secret, or price value is committed.

One command, data kept private

The application ships as a small container image that holds only application code. The confidential data is mounted as a read-only volume at run time, and secrets are provided through the environment. This keeps the image portable and the data private.

```
$ docker compose up --build  
# then open http://localhost:8123/
```

For hosting, a long-running container platform with a persistent private disk for the data and platform-managed secrets is recommended. The health endpoint is used for readiness checks. Serverless platforms are not recommended because the service needs a persistent, private data volume.

Light runtime, hosted intelligence

LANGUAGE Python	WEB SERVICE FastAPI and Uvicorn	PDF PARSING (OFFLINE) pdfplumber
STRUCTURED STORE SQLite	KEYWORD SEARCH BM25 (dependency-free)	SEMANTIC SEARCH OpenAI embeddings, cached to disk
LANGUAGE LAYER OpenAI model, routing and translation only	FRONTEND Self-contained HTML, CSS, JavaScript widget	PACKAGING Docker and Docker Compose

The choice to use hosted embeddings and a hosted model, rather than local machine-learning libraries, keeps the runtime light and avoids heavy local dependencies.

Where everything lives

```
.
├─ data/                LOCAL ONLY, git-ignored: source PDFs and derived data
│   ├── raw/            the client PDFs
│   ├── interim/        intermediate artifacts (manifest, probes)
│   ├── processed/      verified price store, knowledge base, embeddings cache
│   └─ golden/          evaluation baselines
├─ src/fahrschule/      application code
│   ├── taxonomy.py     license-class knowledge
│   ├── store.py        price store and query layer
│   ├── disambiguation.py follow-up decision trees
│   ├── knowledge.py    hybrid FAQ retrieval
│   ├── embeddings.py   semantic index
│   ├── llm.py          bounded language-model adapter
│   ├── dialogue.py     conversation engine (DE and EN, memory)
│   ├── api.py          web service
│   └─ web/index.html   embeddable chat widget
├─ scripts/            the offline data pipeline and the evaluation report
├─ tests/              unit, integration, evaluation, and ground-truth tests
├─ docs/               documentation, build plan, and deploy guide
└─ Dockerfile, docker-compose.yml, requirements.txt
```

Where the project stands

Price sheets integrated and verified	44
Served prices matching their source PDFs	100%
Current variants available	42 current, 2 older archived
Languages	German and English, auto-detected
Follow-up coverage	Every variant of every multi-variant class is reachable
Automated tests and evaluation	80 tests, full evaluation suite, all passing
Deployment	One-command container, data kept private
Adversarial resistance	No input produced a wrong or invented price

Optional, not required for the current scope

- Extract the large truck price-display poster, which uses a different layout.
- Expand the bilingual glossary and add more languages.
- Add a messaging-channel integration (for example WhatsApp) as a thin layer that calls the existing service.
- Restrict the web service to the client's domain in production.
- Add load testing and monitoring dashboards.

Key vocabulary used throughout the documents

Fahrschule	Driving school
Fuehrerschein	Driving license
Klasse	Class (license category)
Grundbetrag	Base fee (covers theory and general costs)
Uebungsstunde	Practice driving lesson
Sonderfahrt	Special drive (overland, motorway, or night)
Pruefung	Exam (theory or practical)
Anhaenger	Trailer
Umschreibung	Conversion of a foreign or older license
Wiedererteilung	Re-issuance of a license after withdrawal
Schaltung / Automatik	Manual / automatic transmission
Oeffnungszeiten	Opening hours
Gesamtbetrag	Grand total